Software Requirements Specification

for

Food Waste Reducer

Version 1.0

Prepared by

Group Name: Team 47

Chalavadi Gayathri SE23UCSE047 se23ucse047@mahindrauniversity.edu.in

Gorrepati Aashish Reddy SE23UCSE070 se23ucse070@mahindrauniversity.edu.in

Gangam Siddanth Reddy SE23UCSE061 se23ucse061@mahindrauniversity.edu.in

Katakam Rutwik SE23UCSE085 se23ucse085@mahindrauniversity.edu.in

Duppeli Varshith SE23UCSE059 se23ucse059@mahindrauniversity.edu.in

Tanishka Guha SE23UARI124 se23uari124@mahindrauniversity.edu.in

P Monisha SE22UARI118 se22uari118@mahindrauniversity.edu.in

Goruputi Pranathi SE23UCSE071 se23ucse071@mahindrauniversity.edu.in

Instructor:	***Ms.* Sai Srithaja Nibhanupudi**
Course:	**CS3201 Software Engineering**
Lab Section:	***IT 2, Ground floor lab***
Teaching Assistant:	***NA***
Date:	**Tentative**

Contents

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
Draft Type and Number	Full Name	Information about the revision. This table does not need to be filled in whenever a document is touched, only when the version is being upgraded.	00/00/00

1 Introduction

The **Food Waste Reducer System** is a web-based application designed to help individuals manage food efficiently and minimize food wastage. Many households experience food waste due to poor tracking of groceries and lack of awareness about expiry dates. This system provides tools that allow users to maintain a digital inventory of their food items, receive alerts when items are close to expiration, generate meal suggestions based on available ingredients, and share excess food with others. By providing these features, the system encourages responsible food consumption and helps users reduce unnecessary waste.

This section of the document provides an overview of the **Software Requirements Specification (SRS)** for the Food Waste Reducer System. It introduces the purpose and scope of the project, explains important definitions and abbreviations used in the document, and outlines the intended audience. The reader will also find references and conventions used throughout the SRS. This introduction serves as a foundation for understanding the system before moving on to the detailed description of system requirements in later sections.

1.1 Document Purpose

This document presents the **Software Requirements Specification (SRS)** for the **Food Waste Reducer System (Version 1.0)**. The purpose of this document is to clearly describe the functional and non-functional requirements of the system so that developers, testers, and stakeholders can understand how the system should operate. It provides a detailed description of the system's features, constraints, and expected behavior. This SRS serves as a reference for the design, development, testing, and maintenance phases of the software development process.

The scope of this document covers the core modules of the Food Waste Reducer System, including **user authentication, food inventory management, expiry alerts, food sharing, meal suggestions, and the analytics dashboard**. These modules together form the primary subsystem of the application that helps users track food items and reduce waste. This document defines what the system should do and outlines the requirements necessary to build and maintain the system effectively.

1.2 Product Scope

The **Food Waste Reducer System** is a web-based application developed to help users manage food inventory efficiently and reduce food wastage. The system allows users to add and track food items, monitor their expiry dates, and receive alerts when items are close to expiring. In addition, the system provides features such as meal suggestions based on available ingredients, and a dashboard that displays analytics about food usage and waste. The application also includes a

food sharing feature that enables users to share excess food with others, helping to prevent unnecessary disposal of usable food.

The primary objective of this system is to promote responsible food consumption and reduce household food waste. By providing a structured way to manage groceries and track expiry dates, the system helps users make better decisions about food usage. The benefits of the system include reducing financial loss due to wasted food, improving food management habits, and supporting sustainability efforts by minimizing food waste. Overall, the Food Waste Reducer System aims to increase awareness and encourage more efficient and environmentally responsible food consumption practices.

1.3 Intended Audience and Document Overview

This document is intended for stakeholders involved in the design, development, and evaluation of the Food Waste Reducer System (FWRS). The primary readers include the professor, who reviews the system requirements and design decisions, and the client or academic stakeholders, who evaluate whether the system meets the intended goal of reducing food waste and improving food inventory management. It is also intended for developers and testers who will use the documented requirements, system features, and use cases as a reference during system design, implementation, and testing.

The Introduction section provides background information, scope, and definitions used throughout the document. Subsequent sections describe the overall system description, functional and nonfunctional requirements, and system interactions. Readers are encouraged to begin with the Introduction and Product Scope sections to understand the context and objectives of the project and then proceed to the system description and requirements sections, which provide detailed specifications necessary for the development and evaluation of the Food Waste Reducer System.

1.4 Definitions, Acronyms and Abbreviations

Term / Acronym	Definition
API	Application Programming Interface – A set of protocols that allows different software components to communicate with each other.
CRD	Create, Read, Update, Delete – The four basic operations performed on data in a database or system.
DBMS	Database Management System – Software used to manage and store system data.

FWRS	Food Waste Reducer System – The software application described in this document that helps users manage food inventory and reduce food waste.
HTML	HyperText Markup Language – The standard markup language used to create web pages.
HTTPS	HyperText Transfer Protocol Secure – A secure version of HTTP used for safe communication over the internet.
JS	JavaScript – A programming language used to implement interactive features in web applications.
REST API	Representational State Transfer Application Programming Interface – A type of web service used for communication between frontend and backend systems.
SRS	Software Requirements Specification – A document that describes the functional and non-functional requirements of a system.
UI	User Interface – The visual elements through which users interact with the system.
URL	Uniform Resource Locator – The address used to access resources on the internet.
UX	User Experience – The overall experience and usability of the system from the user's perspective.
UML	Unified Modeling Language – A standardized modeling language used to visualize system design and architecture.
JSON	JavaScript Object Notation – A lightweight data format used for exchanging information between systems.

1.5 Document Conventions

This document follows the IEEE Software Requirements Specification (SRS) format to ensure clarity, consistency, and ease of understanding for all stakeholders. The following conventions were used while preparing this document.

Formatting Conventions

The document uses Arial font with size 11 or 12 for all text content. All text is single spaced and the document maintains 1-inch margins on all sides. Section and subsection titles follow the numbering format specified in the template to maintain a consistent structure throughout the document. Headings are used to clearly separate different sections of the SRS.

Naming Conventions

All functional requirements in the document are labeled using the format F1, F2, F3, and so on. Performance requirements are labeled using P1, P2, P3, while security and safety requirements are labeled using S1, S2, S3.

Use cases are identified using the format U1, U2, U3, etc., where each identifier corresponds to a specific system interaction.

Typographical Conventions

Important terms, section titles, and system components may be written in bold to highlight their significance. Italic text is used for comments or explanatory notes within the document when necessary. Lists and tables are used where appropriate to improve readability and organize information clearly.

These conventions help maintain consistency throughout the document and ensure that the requirements are easy to understand and reference during system development and evaluation.

1.6 References and Acknowledgments

This section lists the documents, standards, and online resources that were referenced while preparing this Software Requirements Specification (SRS).

1. **IEEE 830-1998 Software Requirements Specification Standard** – Provides guidelines and recommended practices for writing Software Requirements Specification documents.
2. **Unified Modeling Language (UML) Specification** – Used for creating system diagrams such as use case diagrams and other design models.
3. **COMET Method for Software Design** – A software design methodology used for objectoriented analysis and design of software systems.
4. **W3C Web Standards** – Guidelines and standards for web technologies such as HTML, CSS, and JavaScript used in the development of the system.
5. **MDN Web Docs** – Documentation and tutorials for web development technologies including HTML, CSS, and JavaScript.
6. **Draw.io / Lucidchart Documentation** – Tools used for creating UML diagrams and system design diagrams.

These references helped guide the structure, formatting, and technical design considerations used in this SRS document.

2 Overall Description

2.1 *Product Overview*

The **Food Waste Reducer System (FWRS)** is a new, self-contained web-based application designed to help individuals and households manage food more efficiently and reduce unnecessary food waste. The system was developed in response to the growing problem of food wastage caused by poor tracking of groceries and lack of awareness about food expiry dates. Unlike traditional manual tracking methods, this system provides a digital platform where users can maintain a food inventory, receive alerts when food items are nearing expiration, and monitor food usage patterns. The system also introduces additional features such as meal suggestions based on available ingredients, and a food sharing option that allows users to donate excess food.

The FWRS interacts with users through a web interface and relies on browser-based technologies to store and process information. Users input food item details into the inventory, and the system processes this information to generate expiry alerts, analytics, and meal suggestions. The application operates within a browser environment and does not depend on external enterprise systems, making it a standalone solution. However, it can potentially integrate with external services such as grocery databases or mobile applications in future versions.

Product Functionality

2.2 Product Functionality

The Food Waste Reducer System provides the following major functions:

2.3 User Registration and Login: Allows users to create an account and securely log in to the system.

2.4 Food Inventory Management: Enables users to add, view, update, and delete food items in their inventory.

2.5 Expiry Date Tracking: Monitors the expiry dates of stored food items.

2.6 Expiry Alerts: Notifies users when food items are close to expiration so they can use them before they spoil.

2.7 Meal Suggestions: Generates meal ideas based on the ingredients currently available in the user's inventory.

2.8 Food Sharing Feature: Allows users to share or donate excess food to others to prevent waste.

2.9 Food Waste Analytics Dashboard: Displays charts and statistics that show the amount of food used, wasted, or shared.

2.10 Inventory Item Status Management: Allows users to mark food items as used, wasted, or shared, and removes them from the inventory accordingly.

These functions provide users with the necessary tools to manage their food efficiently and encourage responsible consumption. Detailed functional requirements for these features are described in **Section 3 of this document**.

2.11 Design and Implementation Constraints

The development of the **Food Waste Reducer System** is subject to several design and implementation constraints that may limit the options available to developers during the system development process. These constraints arise from the technologies used, development methodology requirements, and system environment limitations.

One of the primary constraints is that the system design must follow the **COMET (Concurrent Object Modeling and Architectural Design Method)** for software engineering. COMET is used to guide the analysis and design of concurrent and distributed systems by identifying system components, interactions, and responsibilities. In addition, the system modeling and documentation must use the **UML (Unified Modeling Language)** to represent system structure and behavior through diagrams such as use case diagrams, class diagrams, and sequence diagrams. These modeling standards ensure that the system design is well-structured, consistent, and easy to understand.

The application will be implemented as a **web-based system**, which means it must operate within the constraints of modern web browsers. The system will be developed using **HTML, CSS, and JavaScript**, and data will be stored using **browser Local Storage** rather than a full-scale database system. This choice simplifies deployment but also limits storage capacity and advanced database features.

Additional constraints include the use of specific libraries and technologies. For example, **Chart.js** will be used to display analytics graphs on the dashboard. The system must also maintain compatibility with common web browsers such as **Google Chrome, Mozilla Firefox, and Microsoft Edge**. From a security perspective, the system must ensure safe handling of user input and prevent unauthorized access to user accounts.

These constraints influence the design choices, development tools, and implementation strategies used during the development of the Food Waste Reducer System.

References

- Gomaa, H. (2011). Software Modeling and Design: UML, Use Cases, Patterns, and Software Architectures. Cambridge University Press. (Reference for the **COMET method**)
- Object Management Group (OMG). Unified Modeling Language (UML) Specification. Available at: https://www.omg.org/spec/UML/ (Reference for **UML modeling language**)

2.13 Assumptions and Dependencies

○ **Internet Access:** It is assumed that users will have access to a stable internet connection when using the web application. ○ **Modern Web Browser Support:** The system assumes that users will access the application using modern web browsers such as Chrome, Firefox, or Edge that support JavaScript and Local Storage. ○ **Accurate User Input:** It is assumed that users will correctly enter food details such as food name, quantity, and expiry date when adding items to the inventory.

○ **Local Storage Availability:** The system assumes that browser Local Storage is available to store user inventory data. If Local Storage is disabled or cleared, stored data may be lost. ○ **Single-User Environment:** The initial version of the system assumes that the application is used by a single user per device and does not require complex multiuser synchronization or cloud database support.

3 Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The **Food Waste Reducer System (FWRS)** provides a web-based user interface that allows users to interact with the system through a standard web browser. The interface is designed to be simple, intuitive, and easy to navigate so that users can efficiently manage their food inventory and monitor food usage.

Users interact with the system through **graphical web pages** that contain menus, buttons, forms, and dashboards. Input is provided using **keyboard, mouse, or touchscreen devices** depending on the user's device. The main interface consists of several pages that allow users to perform different tasks within the system.

The major user interface components include:

- **Login and Registration Page:**

  Allows users to create a new account or log in using their email and password.
- **Dashboard:**
  Displays an overview of the user's food inventory, upcoming expiry alerts, and food waste statistics.

- **Food Inventory Page:**
  Allows users to view all stored food items and perform actions such as adding, updating, or deleting items.
- **Add Food Item Form:**
  A form where users enter details such as food name, quantity, purchase date, and expiry date.
- **Expiry Alerts Page:**
  Displays notifications for food items that are close to their expiration date.
- **Analytics Dashboard:**
  Shows charts and statistics related to food consumption and food waste.

Navigation between pages is performed using a **menu or navigation bar** located at the top of the interface. Buttons and forms guide the user through different operations such as adding food items or updating inventory information.

The interface is designed to be **responsive**, allowing it to adapt to different screen sizes including desktops, laptops, tablets, and mobile devices.

3.1.2 Hardware Interfaces

The Food Waste Reducer System is implemented as a web-based application and does not require any specialized hardware. Users access and operate the system using common computing devices such as:

- Desktop or laptop computers
- Tablets or smartphones with web browsers

3.1.3 Software Interfaces

The **Food Waste Reducer System** interacts with several software components that together enable the system's functionality. These components include the frontend application, backend services, database system, and notification services.

Frontend Application – The frontend of the system is implemented using React (for web). This layer provides the user interface of the system. It is responsible for displaying pages such as the inventory dashboard, expiry alerts, meal planner, and food sharing modules. The frontend captures user inputs, such as adding food items or viewing alerts, and sends requests to the backend server through REST API calls.

Backend Application – The backend is implemented using Node.js with the Express framework. This layer contains the core business logic of the system. It processes user requests received from the frontend, performs validation checks, manages food inventory data, calculates expiry alerts, and handles food-sharing operations. The backend acts as the communication bridge between the frontend interface and the database system.

Database System – The system uses MongoDB as its primary database management system. MongoDB is a NoSQL database that stores data in a flexible document-based format. The database stores persistent information required by the application, including:

- User account information
- Food inventory details (food name, quantity, purchase date, expiry date)
- Meal planning data
- Food sharing records
- Food usage and waste statistics

The backend communicates with MongoDB using database queries to store, retrieve, update, and delete system data.

Notification Service – The system uses Firebase Cloud Messaging (FCM) to deliver expiry alerts and reminders to users. Firebase enables the application to send real-time notifications when food items are close to expiration or when important updates occur.

Communication between the frontend and backend occurs through REST API endpoints, while the backend interacts with the MongoDB database and Firebase services to manage data and notifications within the Food Waste Reducer System.

3.2 Functional Requirements

F1: User Registration
The system shall allow new users to create an account by providing basic information such as name, email address, and password.

F2: User Login
The system shall allow registered users to log in securely using their email and password.

F3: Add Food Item
The system shall allow users to add food items to their inventory by entering details such as food name, quantity, purchase date, and expiry date.

F4: View Food Inventory

The system shall display a list of all food items currently stored in the user's inventory along with their expiry dates.

F5: Update Food Item Information

The system shall allow users to edit or update information about existing food items in the inventory.

F6: Delete Food Item

The system shall allow users to remove food items from the inventory when they are used, wasted, or no longer needed.

F7: Expiry Date Monitoring

The system shall continuously monitor the expiry dates of food items stored in the inventory.

F10: Expiry Alerts and Notifications

The system shall notify users when food items are approaching their expiry dates through alerts or notifications.

F11: Meal Suggestions

The system shall generate meal suggestions based on the food items currently available in the user's inventory.

F12: Food Sharing

The system shall allow users to share or donate excess food with other users through the food sharing feature.

F13: View Food Waste Analytics

The system shall display analytics and statistics showing the amount of food used, wasted, or shared.

F14: Mark Food Status

The system shall allow users to mark food items as **used, wasted, or shared** for tracking and analytics purposes.

F15: Search and Filter Inventory

The system shall allow users to search and filter food items in the inventory based on name or expiry date.

3.3 Use Case Model

3.3.1 Use Case #1 – User Login & Authentication (U1)

Author: Gayathri C

Purpose:
Provides users with an overview of their food inventory, alerts, and meal plans in one place.

Requirements Traceability: F1 – View Dashboard

Priority: High

Preconditions: User must be logged into the system.

Postconditions: Dashboard information is displayed.

Actors: User

Extends: None

Flow of Events :

Basic Flow

1. User logs into the system.
2. System loads dashboard.
3. System displays inventory summary, alerts, and meal plans.

Alternative Flow

User refreshes dashboard.

Exceptions

System fails to retrieve data.

Includes

U3 Food Inventory U4
Expiry Alerts

Notes

Dashboard should update automatically when data changes.

3.3.2 Use Case #2 – Meal Planner (U2)

Author: *Tanishka Guha*

Purpose:
Allows users to plan meals based on available ingredients to reduce food waste.

Requirements Traceability: F2 – Meal Planning

Priority: High

Preconditions: User must be logged in.

Postconditions: Meal plan is generated or saved.

Actors: User

Extends: None

Flow of Events

Basic Flow

1. User opens Meal Planner page.
2. System retrieves available ingredients.
3. User selects or generates meal suggestions.
4. System saves meal plan.

Alternative Flow

User edits meal plan.

Exceptions

Ingredients not available.

Includes

U3 Food Inventory

Notes

Future versions may integrate recipe APIs.

3.3.3 Use Case #3 – Food Inventory (U3)

Author: *Rutwik Katakam*

Purpose: Allows users to track food items and expiry dates.

Requirements Traceability: F3 – Manage Food Inventory

Priority: High

Preconditions: User must be logged in.

Postconditions: Inventory list is updated.

Actors: User

Extends: None

Flow of Events

Basic Flow

1. User opens Inventory page.
2. User adds or edits food items.
3. System saves inventory data.

Alternative Flow

User deletes an item.

Exceptions

Invalid data entry.

Includes

None

3.4.4 Use Case #4 – Expiry Alerts (U4)

Author: *Varshith Duppeli*

Purpose: Notifies users when food items are close to their expiration date.

Requirements Traceability: F4 – Expiry Notifications

Priority: High

Preconditions: Food items must have expiry dates stored.

Postconditions: User receives notification.

Actors: User

Extends: None

Flow of Events

Basic Flow

1. System checks expiry dates.
2. System identifies items nearing expiry.
3. System sends alerts to the user.

Alternative Flow

User disables alerts.

Exceptions

Notification system fails.

Includes

U3 Food Inventory

Notes

Firebase notifications may be used.

3.4.5 Use Case #5 – Storage Tips (U5)

Author: *Monisha paidy*

Purpose: Provides users with tips on how to properly store food to extend shelf life.

Requirements Traceability: F5 – Food Storage Tips

Priority: Medium

Preconditions: User must access tips page.

Postconditions: Storage recommendations are displayed.

Actors: User

Extends: None

Flow of Events

Basic Flow

1. User opens Storage Tips page.
2. System displays storage recommendations.

Alternative Flow

User searches for specific food storage tips.

Exceptions

Content not available.

Includes

None

Notes

Tips may be sourced from food safety databases.

3.4.6 Use Case #6 – Local Programs (U6)

Author: Gorrepati Aashish Reddy

Purpose: Helps users find nearby food-sharing or composting programs.

Requirements Traceability: F6 – Find Local Programs

Priority: Medium

Preconditions: User must allow location access.

Postconditions: Nearby programs are displayed.

Actors: User

Extends: None

Flow of Events

Basic Flow

1. User opens Local Programs page.
2. System retrieves nearby programs.
3. System displays program information.

Alternative Flow

User manually enters location.

Exceptions

Location services unavailable.

Includes

None

Notes

Integration with map services may be required.

3.4.7 Use Case #7 – Food Sharing (U7)

Author: Gangam Siddanth Reddy

Purpose: Allows users to share excess food with others in their community.

Requirements Traceability: F7 – Food Sharing

Priority: Medium

Preconditions: User must be logged in.

Postconditions: Food sharing request is created.

Actors: User

Extends: None

Flow of Events

Basic Flow

1. User selects food item.
2. User chooses share option.
3. System posts sharing information.

Alternative Flow

User cancels sharing.

Exceptions

Network failure.

Includes

U3 Food Inventory

Notes

Community verification may be required.

3.3.8 Use Case #8 – Admin Panel (U8)

Author: Goruputi Pranathi

Purpose: Allows administrators to manage users, food data, and reports.

Requirements Traceability: F8 – System Administration

Priority: High

Preconditions: Admin must log into the admin panel.

Postconditions: System data is updated.

Actors: Admin

Extends: None

Flow of Events

Basic Flow

1. Admin logs into the admin panel.
2. Admin views system data.
3. Admin updates or deletes records.

Alternative Flow

Admin generates reports.

Exceptions

Unauthorized access attempt.

Includes

None

Notes

Admin access must be secured.

4 Other Non-functional Requirements

4.1 Performance Requirements

P1. User Authentication Response Time
The system shall authenticate a user during the login process within 3 seconds after valid credentials are submitted.

P2. Food Inventory Update Time
The system shall add, update, or delete a food item in the inventory within 2 seconds after the user submits the request.

P3. Page Loading Time
The system shall load main application pages such as the dashboard, inventory page, and analytics page within 3 seconds under normal network conditions.

P4. Expiry Alert Processing
The system shall check the food inventory for items approaching their expiry date at least once every 24 hours and generate alerts accordingly.

P5. Notification Delivery
Expiry alerts and reminders sent through the notification service shall reach the user within 5 seconds after they are triggered by the system.

P6. Analytics Dashboard Generation
The system shall generate and display food waste analytics and statistics within 4 seconds after the user opens the analytics dashboard.

P7. Concurrent User Support
The system shall support at least 100 simultaneous users performing operations such as login, inventory management, and viewing analytics without noticeable performance degradation.

4.2 Safety and Security Requirements

Safety and security requirements ensure that the Food Waste Reducer System (FWRS) protects user data, prevents unauthorized access, and maintains reliable system operation. These requirements focus on minimizing risks related to data loss, misuse of the system, and privacy violations.

S1. User Authentication Security

The system shall require users to authenticate their identity using a valid email and password before accessing personal account information and system features.

S2. Secure Password Storage

The system shall store user passwords in an encrypted or hashed format to prevent unauthorized access to sensitive information.

S3. Secure Communication

All communication between the user's device and the backend server shall occur through secure HTTPS connections to protect data transmitted over the internet.

S4. Email Verification

The system shall require **email verification** during account registration and password reset processes to confirm user identity and prevent unauthorized account creation.

S5. Access Control

The system shall ensure that users can only access and modify their own food inventory data and personal information.

S6. Data Privacy Protection

The system shall protect user data such as email addresses, food inventory information, and usage statistics from unauthorized access or disclosure.

S7. Secure Notification Handling

Notifications related to expiry alerts or reminders shall not expose sensitive user data and shall only contain necessary information.

S8. Account Recovery Protection

The system shall provide secure password reset functionality through verified email links that expire after a limited period.

S9. Data Backup and Recovery

The system shall periodically back up important system data to prevent permanent data loss in case of server failure or technical issues.

S10. Session Security

User sessions shall automatically expire after a period of inactivity to reduce the risk of unauthorized access.

4.3 Software Quality Attribute

4.3.1 Reliability

The system shall provide reliable operation during normal usage conditions. The application shall ensure that user actions such as adding, updating, or deleting food items are completed successfully without data loss. The system shall also validate all user inputs before processing them to prevent

errors or incorrect data storage. In case of unexpected failures, the system shall display appropriate error messages and allow the user to retry the operation.

4.3.2 Usability

The system shall provide a user-friendly interface that allows users to easily manage their food inventory and access system features. The interface shall use clear menus, buttons, and labels so that users can understand the functionality without requiring extensive training. Navigation between pages such as login, dashboard, inventory, and analytics shall be simple and intuitive. The system shall also provide clear feedback messages when actions such as adding or deleting food items are performed.

4.3.3 Maintainability

The system shall be designed in a modular manner so that individual components such as the user interface, inventory management module, and notification system can be modified or updated independently. This approach will make it easier for developers to maintain and improve the system in the future. The code shall follow standard programming conventions and include proper documentation to simplify debugging and system updates.

4.3.4 Portability

The Food Waste Reducer System shall be accessible through standard web browsers such as Chrome, Firefox, and Edge. The system shall operate on different devices including desktop computers, laptops, tablets, and smartphones without requiring additional software installation. This ensures that users can access the system from multiple platforms.

4.3.5 Flexibility

The system shall be designed to support future enhancements such as integration with external food donation platforms, or advanced analytics tools. The architecture will allow developers to add new features without major changes to the existing system structure.

5 Other Requirements

*<This section is **Optional**. Define any other requirements not covered elsewhere in the SRS. This might include database requirements, internationalization requirements, legal requirements, reuse objectives for the project, and so on. Add any new sections that are pertinent to the project.>*

Appendix A – Data Dictionary

The following tables describe the main **data entities, variables, inputs, outputs, and operations** used in the system.

- User Data Elements

Data Element	Type	Description	Operations
Operations	Integer	Unique identifier assigned to each user in the system	Used for authentication and database indexing
name	String	Full name of the registered user	Used for user profile display
email	String	Email address of the user	Used for login, authentication, and notifications
password	String	Encrypted password used for user authentication	Used during login and account verification
location	String	Geographic location of the user	Used to find nearby food sharing or composting programs

- Food Item Data Elements

Data Element	Type	Description	Operations
foodId	Integer	Unique identifier for each food item	Used for item management in inventory
name	String	Name of the food item	Displayed in food inventory
quantity	Integer	Amount of the food item stored	Used for tracking available food
expiryDate	Date	Expiration date of the food item	Used to generate expiry alerts

category	String	Category of food (e.g., dairy, vegetables, fruits)	Used for classification and storage tips

- Meal Plan Data Elements

Data Element	Type	Description	Operations
mealPlanId	Integer	Unique identifier for each meal plan	Used to manage user meal plans
date	Date	Date associated with the meal plan	Used for scheduling meals
foodItems	List	List of food items included in the meal plan	Used to reduce food waste through planning

- Notification Data Elements

Data Element	Type	Description	Operations
notificationId	Integer	Unique identifier for notifications	Used for tracking notifications
message	String	Notification message sent to the user	Displayed as alerts
date	Date	Date the notification was generated	Used for notification history
status	String	Indicates whether notification is read or unread	Helps manage alert tracking

- Local Program Data Elements

Data Element	Type	Description	Operations
programId	Integer	Unique identifier for local programs	Used to track available programs
name	String	Name of the food sharing or composting program	Displayed to users
location	String	Location of the program	Used to find nearby services

type	String	Type of program (food sharing or composting)	Used for filtering services

- Storage Tip Data Elements

Data Element	Type	Description	Operations
tipid	Integer	Unique identifier for storage tips	Used to track storage recommendations
foodType	String	Types of food associated with the tip	Used to match tips with food items
tipDescription	String	Description of proper food storage tip	Displayes to help extend shelf life

- System Inputs

Input	Description
User Registration Details	User enters name, email, password, and location during signup
Food Item Information	User enters food name, quantity, category, and expiry date
User Meal Planning Data	User selects food items to create meal plans
Location Data	Used to find nearby food sharing or composting programs

- System Outputs

Output	Description
Expiry Alerts	Notifications generated when food items approach their expiry date
Storage Tips	Recommendations on how to store specific food items
Meal Plan Suggestions	Suggested meal plans based on available food items

Local Program Recommendations	Display of nearby food sharing or composting services

Appendix B - Group Log

Activity	Description	Date
Project Kickoff	Initial discussion on project objectives, scope, and requirements. Roles were assigned among group members.	6 February
Requirement Analysis	Discussion on system features including meal planning, food inventory tracking, expiry alerts, and storage tips.	10 February
UI Design Planning	Brainstorming session for designing the user interface and application layout. Wireframes were prepared.	15 February
Frontend Development Discussion	Review of frontend implementation progress using React/Flutter and adjustments to UI components.	20 February
Backend Architecture Planning	Discussion on backend design using Node.js and Express framework. API structure and database schema were finalized.	5 March
Database Implementation Review	MongoDB collections and data models were designed	20 March
	and integrated with the backend.	

Notification System Integration	Integration of Firebase Cloud Messaging to implement expiry alerts and reminders.	5 April
API Integration	Integration of third-party APIs including Google Maps API for locating nearby food sharing programs.	15 April
System Integration Meeting	Frontend and backend components were integrated and tested for functionality.	25 April
Testing and Bug Fixing	System testing was performed to identify and resolve bugs.	5 May
Final Submission Preparation	Final review of the system, documentation preparation, and submission of the completed project.	10 May